BPTT got hands

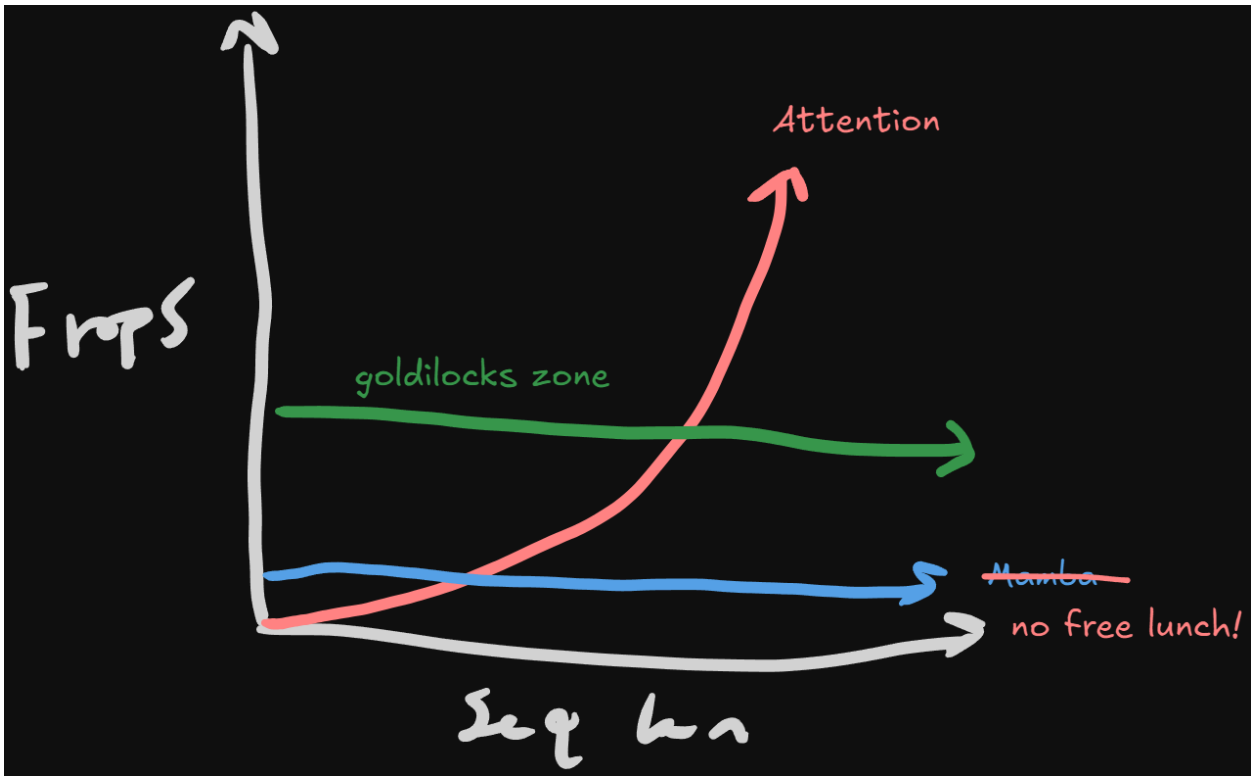
This article is a null result
Backpropagation 3 - _ueaj 1

I'm working towards my 'holy grail', which I outlined in 'Cognitronium' but from an architectural approach. I want to do this for four reasons:

1. I want it to be continuous simply because it feels more cognitronium shaped, the jagged boundaries of context summarization don't feel sufficiently soup like
2. I'm not in a position to take advantage of near-term economically useful AI anyways, and I like dreaming big so I wouldn't be entertained with short term solutions anyways
3. It effectively removes the context window limit, so even if we do context summarization cognitronium, we have more wiggle room to work with.
4. There are issues with attention that prevent it from being truly viable on long context that we need to solve anyways, as noted by mike, and others and empirically validated by benchmarks

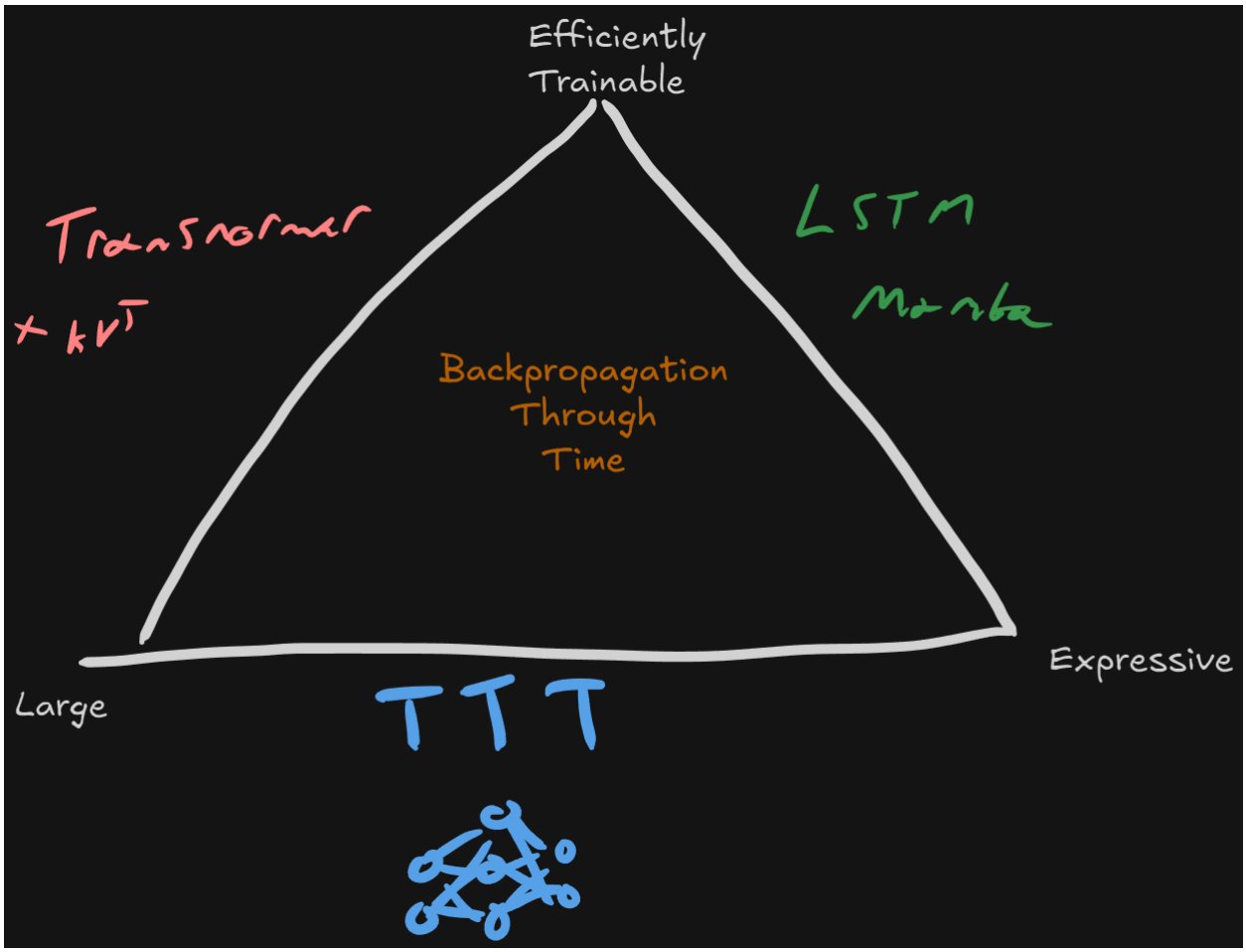
Designing good RNNs

However, I've never really believed in the O(1) of SSM/RNN inference, and after mamba's failure to dethrone the transformer I feel vindicated. That being said, the answer to this is not to abandon O(1) inference, but to abandon the "free lunch".



The idea is simple, amortize the flops/token over some fixed sequence length. If there isn't a free lunch, then we should prioritize having large, expressive hidden states. This idea has already been explored by xLSTM and Transnormer (my personal favorite). However there is a problem: you can't have a hidden state that is both large, expressive **and** trainable.

BPTT's Impossible Triangle



- If you choose a recurrence who's update rule is linear (like transnormer), then your recurrence isn't expressive, and won't beat the transformer in the real world
- If you try to make an recurrence who's update rule is non-linear, then you need to use backpropagation through time (BPTT) to backpropagate the error signals through the sequence length. This means you need to use checkpointing across the sequence length, which limits the size of your hidden state, thus it can't be large.
- If you have a recurrence that is large and expressive, then you either need lots of checkpointing or to recompute the state for every time step, requiring $O(n^2)$ training compute on an already flop-intensive amortized complexity $O(kn^2) \approx O(n^3)$!!

The weakness in the link here is BPTT. If we can find a way to train an expressive update rule without checkpointing, then we can use a TTT layer or similar system as our hidden state. We can make it as big as we want, maybe we can even fit the whole model in the recurrent state!

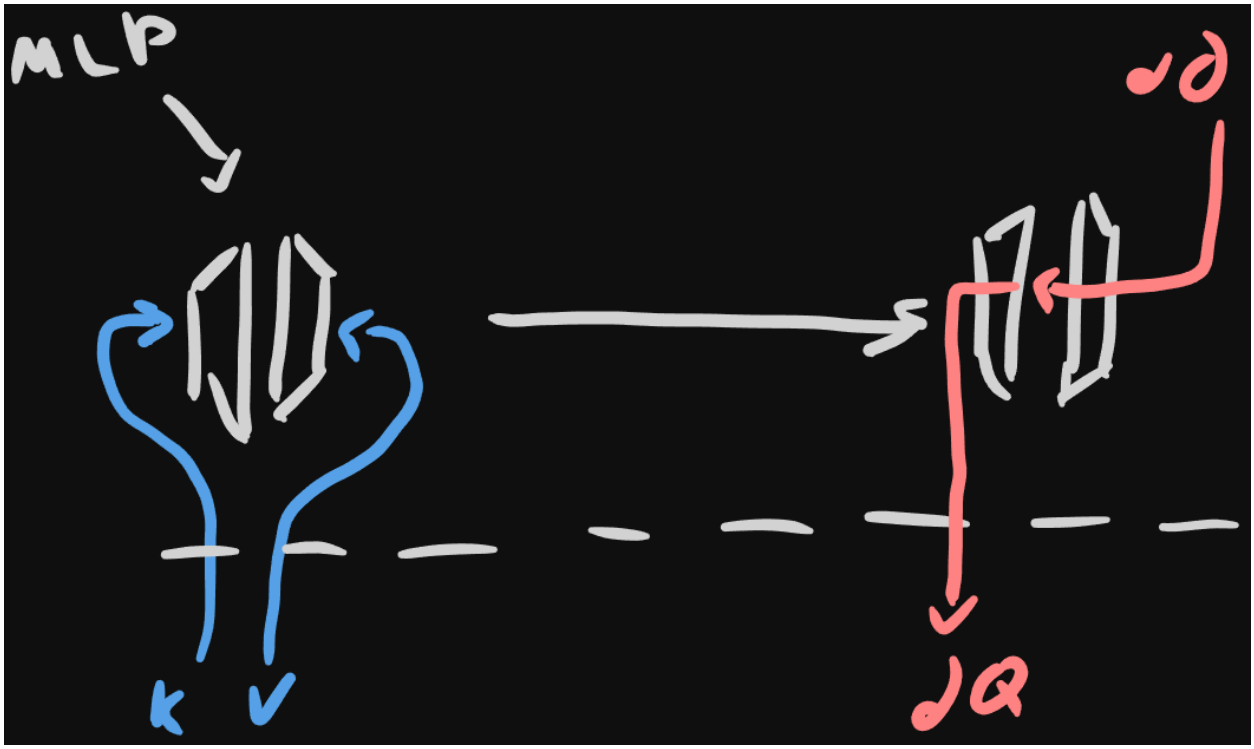
Test time training

Test time training is a pretty simple idea: have a little model be your recurrent hidden state. Your main model spits out vector pairs that this model is trained on across the sequence length. It's very effective, and has been successfully applied in a number applications. I think some ARC-AGI attempts used it as well.

The low MFU issues aren't real and you can solve it with multiheaded, batched and multilayer TTT and with kernels. Each token outputs a batch, multiple token batches are aggregated together into a bigger batch, and to make up for the lower training pair size per token, you can use multiple layers. If doing multi-device training, make each device's TTT module independent, i.e. 4 parallel TTT layers.

Slaying the dragon

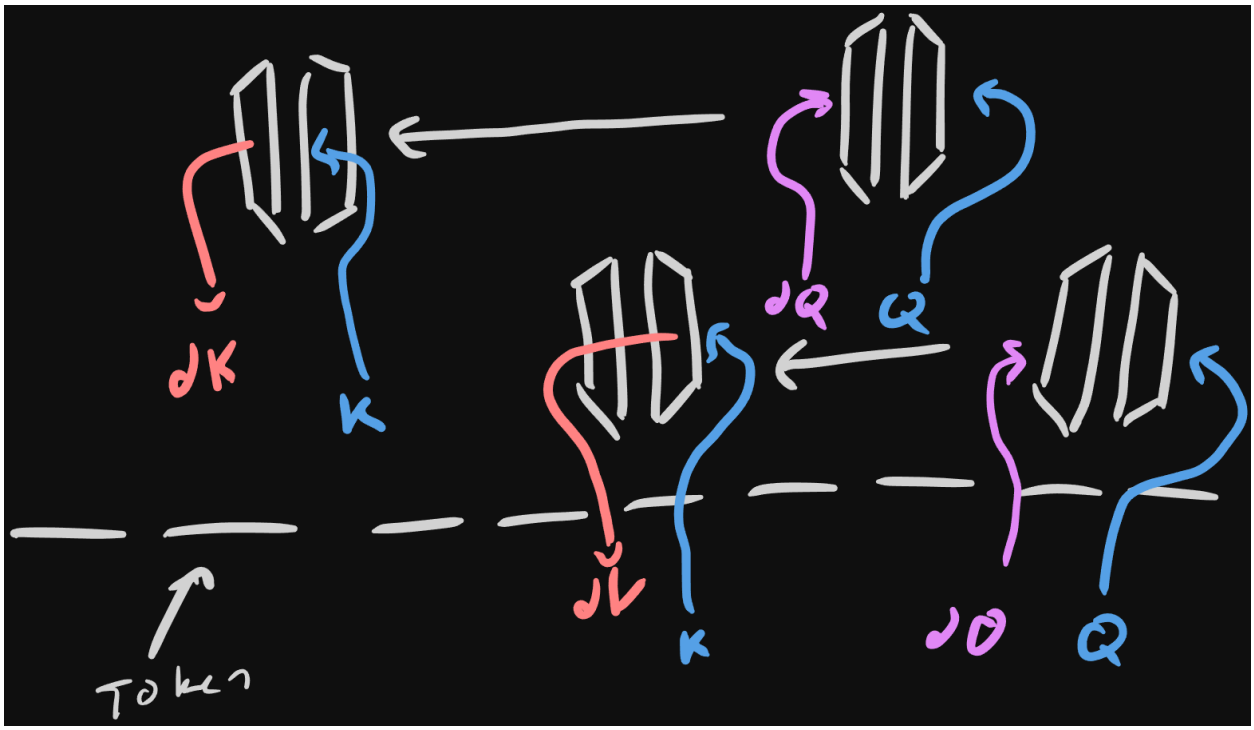
However, as mentioned before, it requires BPTT to work. This constrains the size of the hidden state, so to solve this I thought of a surrogate gradient method to try and minimize this.



The first step is to calculate the query gradient exactly, and the hidden state gradient nearly exactly (up to non-associativity of floating point operations). This pass runs forward through the sequence, taking the dO and differentiating it through the current hidden state to obtain the dQ.

The second is to calculate the key and value gradients approximately. We do this by running through the sequence **backwards** and using the final hidden state of the forward pass (with all the output projection heads re-initialized to zero), where we TRAIN the hidden state to compress the relationship between the query and output gradient for the value gradient. We train an additional module to compress the relationship between the query and the query gradient.

Then as we go backwards through the sequence, we recall the relationships compressed in the hidden state to approximate the gradient. You pass in the key to get out the value gradient and the key for the query gradient.

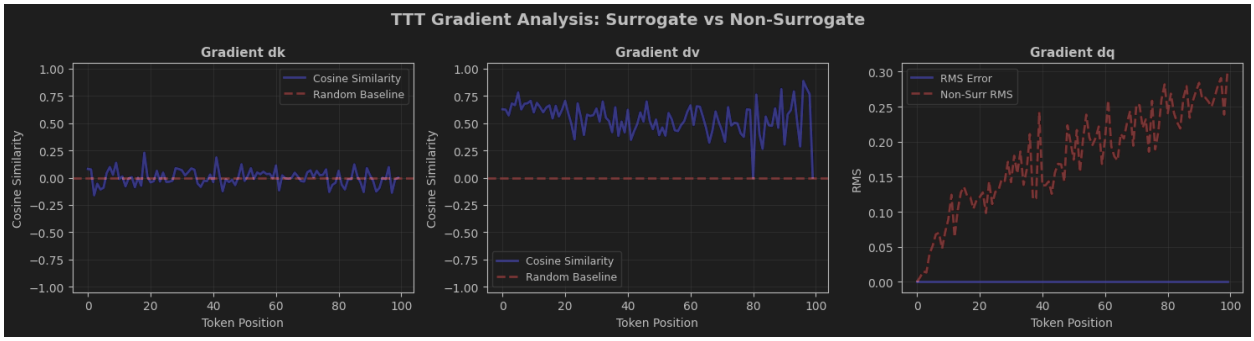


I know what you're thinking.
This sounds like bullshit.

Results

Well you'd be right. It doesn't work. :P

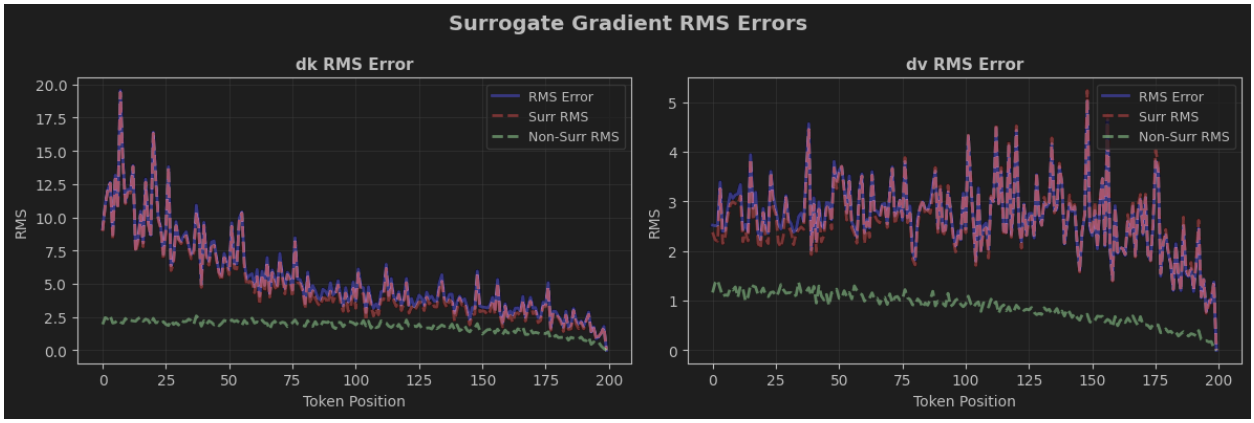
Well kinda, it actually works for the query, value and hidden state gradients just fine! The hidden state computation needs to be done in fp32 to account for non-associativity of floating point addition and the query gradients are the exact gradients (they are not approximate).



As all of you are surely familiar, convergence under gradient descent is still guaranteed as so long as your search direction is $\vec{d}^T\vec{g} > 0$. And as you can see, the value gradient is in fact reliably above 0. This works for ~500 tokens on purely random data, fully trained information is more predictable, so in practice it might work better. Additionally you can see the query gradients are exactly correct, we plot the RMS error there because it's a stronger criteria that also works.

I tried all sorts of things for dk to get it to work, including treating the transpose of the hidden state module... as a module! My thinking was what if we trained the backwards pass through the model itself to compress the key gradients?? While JAX (my beloved) made that easy, it still didn't work. I also tried switching up some of the relationships, etc. none of it worked unfortunately.

We interrupt this article to shill for JAX. If you're doing TTT or OOD research like this, it's way easier in JAX. The logic for this extremely unorthodox surrogate gradient method was only 100 lines of code thanks to JAX's modularity and composability. Code is posted in the conclusion.



The overall RMS error is still high for dv, but a bit of normalization or hparam tuning can get the magnitudes right, and as so long as the search direction is valid, the model will still train.

Conclusion

This is actually my fourth time fighting backpropagation so far. I had 2 private attempts + one here and this one. Only one of the private ones worked, though it was very similar to a GDM paper that beat me to it. There was some differences that I want to revisit at a later date but this is lower down on my priority list.

I think the approach I took in this article could work, a more persistent researcher could probably figure it out. I'm not super interested in taking this any further simply because the GDM

I really need to do that RL project, thanks to @mike64_t for nerdsnipping me on recurrence.

Code

This is the file that implements the TTT method. Below this is a non-batched Flax NNX module so you can put it in model. The code can also be found on github here

```
import jax
import jax.numpy as jnp
import functools as fn

def make_reverse_fn(fwd_fn):
    def run_vjp(state, x):
        q, do = x
        o, dq_fn = jax.vjp(fn.partial(fwd_fn, state), q)
        return dq_fn(do)[0]
    return run_vjp


def make_scan_fn(fwd_fn, distance=True, n_iters=1, wd=.1, lr=.01):
    def update_fn(state, x):
        k, v, q = x
        o = fwd_fn(state, q)

        for _ in range(n_iters):
            v_pred, dstate_fn = jax.vjp(lambda state: fwd_fn(state, k),
state)
            dv = (v - v_pred) if distance else v
            dstate, = dstate_fn(dv)

            # todo optimizer
            state = jax.tree.map(lambda a, b: (1-wd*lr)*a +
lr*jax.nn.tanh(b), state, dstate)

        return state, o
    return update_fn

def ttt(fwd_fn, surrogate=True):
    fwd_scan = make_scan_fn(fwd_fn, distance=True)

    def _ttt_fwd(k: jax.Array, v: jax.Array, q: jax.Array, state):
        k_seq, v_seq, q_seq = k.swapaxes(0, 1), v.swapaxes(0, 1),
q.swapaxes(0, 1)

        new_state, o_seq = jax.lax.scan(fwd_scan, state, (k_seq, v_seq,
q_seq))

        o = o_seq.swapaxes(0, 1)
        if surrogate:
            return o, (k_seq, v_seq, q_seq, state)
```

```
        else:
            return o

    if not surrogate:
        return _ttt_fwd

    @jax.custom_vjp
    def ttt_inner(k: jax.Array, v: jax.Array, q: jax.Array, state):
        return _ttt_fwd(k, v, q, state)[0]

    v_scan = make_scan_fn(fwd_fn, distance=False)
    # Train a backwards pass module
    # k_fwd_fn = make_reverse_fn(fwd_fn)
    # k_scan = make_scan_fn(k_fwd_fn, distance=False)
    # Regular module for key
    k_scan = make_scan_fn(fwd_fn, distance=True)
    def _ttt_bwd(res, do):
        k_seq, v_seq, q_seq, state = res
        do_seq = do.swapaxes(0, 1)

        def q_scan(carry, x):
            k, v, q, do = x
            state, dstate = carry

            (new_state, o), q_update_jvp = jax.vjp(lambda state, q:
fwd_scan(state, (k, v, q)), state, q)
            new_dstate, dq = q_update_jvp((dstate, do))

            return (new_state, new_dstate), (o, dq)

        dstate = jax.tree.map(jnp.zeros_like, state)

        (end_state, dstate), (o_seq, dq_seq) = jax.lax.scan(q_scan,
(state, dstate), (k_seq, v_seq, q_seq, do_seq))

        def kv_scan(carry, x):
            k_state, v_state = carry
            k, v, q, o, do, dq = x

            new_v_state, dv = v_scan(v_state, (q, do, k))
            # new_k_state, dk = k_scan(k_state, ((q, o+do), dq, (k,
v+dv)))
            new_k_state, dk = k_scan(k_state, (q, q+dq, k))
```

```
            return (new_k_state, new_v_state), (dk, dv)

        # todo no special case
        k_state = jax.tree.map(lambda x: x, end_state)
        k_state.down_proj = jax.tree.map(jnp.zeros_like,
k_state.down_proj)

        v_state = jax.tree.map(lambda x: x, end_state)
        v_state.down_proj = jax.tree.map(jnp.zeros_like,
v_state.down_proj)

        (_, _), (dk_seq, dv_seq) = jax.lax.scan(kv_scan, (k_state,
v_state), (k_seq, v_seq, q_seq, o_seq, do_seq, dq_seq), reverse=True)

        dq, dk, dv = dq_seq.swapaxes(0, 1), dk_seq.swapaxes(0, 1),
dv_seq.swapaxes(0, 1)
        return dk, dv, dq, dstate

    ttt_inner.defvjp(_ttt_fwd, _ttt_bwd)
    return ttt_inner
```

Flax NNX module, not required

```
from typing import Optional, Callable

import jax
import jax.numpy as jnp
from flax import nnx
from flax.nnx import rnglib as rng

from ueaj.model import GMLP
from ueaj.model.einsum import Einsum, lecun_normal_init, zeros_init
from ueaj.utils.configurator import config
from .impl import ttt


@config
class TTTModel(nnx.Module):
    """Test-Time Training layer that learns to adapt its hidden state
during inference.

    The TTT layer maintains a hidden state that is updated at each
```

```
sequence position    using gradient descent on a self-supervised
objective. The state is used to    produce outputs via an inner
learnable module.
    Args:       model_d: Model dimension (input/output dimension)
hidden_d: Hidden dimension for the state (defaults to model_d)
module: Inner module class to use as fwd_fn (default: GMLP)
module_kwargs: Additional kwargs to pass to the inner module
param_dtype: Parameter dtype       surrogate: Whether to use surrogate
gradients (custom VJP) for backprop       rngs: Random number generators
mesh: Optional JAX mesh for distributed training    """    def __init__(
        self,
        model_d: int,
        hidden_d: int | None = None,
        module: Callable = GMLP,
        module_kwargs: dict | None = None,
        param_dtype: jnp.dtype = jnp.bfloat16,
        surrogate: bool = True,
        *,
        rngs: rng.Rngs,
        mesh: Optional[jax.sharding.Mesh] = None
    ):
        super().__init__()

        if hidden_d is None:
            hidden_d = model_d

        if module_kwargs is None:
            module_kwargs = {}

        self.model_d = model_d
        self.hidden_d = hidden_d
        self.surrogate = surrogate

        # Create fused k, v, q projection
        size_dict = {'d': model_d, 'h': hidden_d, 'i': 3}
        self.kvq_proj = Einsum(
            "bnd,idh->ibnh",
            size_dict=size_dict,
            batch_dims="i",
            rngs=rngs,
            dtype=param_dtype,
            mesh=mesh,
            sharding=(None, None, 'tensor') if mesh is not None else None
```

```
        )

        # Create inner module - its parameters will be the TTT state
        # This module takes (batch, seq, hidden_d) input and produces
(batch, seq, hidden_d) output
        self.inner_module = module(
            model_d=hidden_d,
            rngs=rngs,
            mesh=mesh,
            **module_kwargs
        )

        # Output projection to map from hidden_d back to model_d
        size_dict_out = {'d': model_d, 'h': hidden_d}
        self.out_proj = Einsum(
            "bnh,hd->bnd",
            size_dict=size_dict_out,
            initializer=zeros_init,
            rngs=rngs,
            dtype=param_dtype,
            mesh=mesh,
            sharding=('tensor', None) if mesh is not None else None
        )

        # Create the TTT forward function
        self.ttt_fn = ttt(self._fwd_fn, surrogate=surrogate)
        self.inner_module_gdef = nnx.graphdef(self.inner_module)

    def _fwd_fn(self, module_state: nnx.State, x: jax.Array) ->
jax.Array:
        """Forward function for TTT: reconstructs module from state and
applies it.

        Args:           module_state: NNX State containing the module's
parameters (this is the TTT state)           x: Input of shape (batch
```